WildBook Eco-Tourism Platform

COMPLETE TECHNICAL ARCHITECTURE, FILE DIRECTORY & VIVA DEFENSE MANUAL (9-TABLE STREAMLINED SCHEMA)

> **Document Purpose & Academic Defense Summary**
>
> This reference document provides an exhaustive, step-by-step breakdown of the entire WildBook project. It details the **9-Table Production Schema**, mathematical proofs of normal forms (1NF, 2NF, 3NF, BCNF), justification for consolidating 5 redundant tables without losing any feature or business logic, backend controller mapping, frontend component architecture, and line-by-line implementation of core business logic (preventing double-bookings, row-level locking with `SELECT ... FOR UPDATE`, 10-minute hold expiration, capacity triggers, and the 40%/60% cancellation split).

1. Relational Database Schema & Normalization Analysis

The WildBook database is implemented on MySQL (InnoDB Engine) and strictly adheres to relational design patterns across **9 normalized tables**:

#	TABLE NAME	PRIMARY KEY	FOREIGN KEYS	RELATIONAL PURPOSE & DOMAIN ENTITY
1	`users`	`user_id`	`reviewed_by → users`	Core identity table for all 3 system roles (`VISITOR` , `ADMIN` , `SITE_OWNER`) with bcrypt hashed credentials.
2	`admin_profiles`	`user_id`	`user_id → users`	1:1 operator profile storing credentials, registered address, and document file paths (`license_doc` , `identity_doc` , `auth_letter_doc`).
3	`sanctuaries`	`sanctuary_id`	`admin_id, reviewed_by → users`	Master reserve entity containing geo coordinates, operating hours, `cover_image` , and rating metrics (`average_rating` , `total_reviews`).
4	`activities`	`activity_id`	`sanctuary_id → sanctuaries`	Safari types (4x4 Jeep Safari, Boat Safari, Canopy Walks) with duration and pricing.
5	`guides`	`guide_id`	`sanctuary_id → sanctuaries`	Certified naturalists and forest department accredited wildlife trackers.
6	`vehicles`	`vehicle_id`	`sanctuary_id → sanctuaries`	Authorized safari transport fleet (4x4 Gypsies, cruisers, safari boats) with certified physical capacities.
7	`slots`	`slot_id`	`activity_id, guide_id, vehicle_id`	Scheduled departure instances linking activity, guide, vehicle, date, and time. Protected by compound UNIQUE constraints.
8	`bookings`	`booking_id`	`slot_id → slots, user_id → users`	Visitor reservation master record with seat count (`number_of_people`), 10-minute hold timestamp, and integrated cancellation tracking (`cancelled_at` , `cancellation_reason` , `cancellation_fee` , `refund_amount`).
9	`payments`	`payment_id`	`booking_id → bookings`	Financial transaction ledger storing amount, status (`PENDING` , `PAID` , `PARTIALLY_REFUNDED` , `FAILED`), transaction reference, and timestamp.

How the Schema Was Reduced from 14 Tables to 9 Tables (Viva Defense)

Examiners frequently ask: *"Why did you reduce from 14 tables down to 9 tables, and does this violate normalization?"* Here is the exact mathematical and structural proof to defend in your viva:

1. **Consolidation of `cancellations` into `bookings` (Saved 1 Table)**
   Why: A cancellation is a 1-to-1 lifecycle state transition of a booking. A booking can have at most one cancellation event. In relational algebra, attributes `cancelled_at` , `cancellation_reason` , `cancellation_fee` , and `refund_amount` depend directly and solely on `booking_id` (the primary key).
   Normalization: **Strictly 3NF and BCNF compliant** because there are no transitive dependencies (–> no non-prime attribute depends on another non-prime attribute).
2. **Consolidation of `admin_documents` into `admin_profiles` (Saved 1 Table)**
   Why: In the government reserve onboarding workflow, an administrator is mandated to submit exactly three discrete verification documents: Operating License, Photo Identity Proof, and Authorization Letter. Storing these as `license_doc` , `identity_doc` , and `auth_letter_doc` inside `admin_profiles` stores atomic file paths.
   Normalization: **Strictly 1NF, 2NF, and 3NF compliant** because each attribute represents an atomic scalar value and functionally depends on the candidate key `user_id` .

3. **Consolidation of `sanctuary_images` into `sanctuaries` (Saved 1 Table)**
   Why: In real-world reserve discovery, the primary visual asset is the verified `cover_image`. Storing `cover_image VARCHAR(255)` directly on `sanctuaries` eliminates an unnecessary join table while ensuring instantaneous card rendering on the frontend.
   Normalization: **Strictly 3NF compliant** since the cover photo URL is a single-valued attribute of the sanctuary entity.
4. **Consolidation of `reviews` into `sanctuaries` (Saved 1 Table)**
   Why: Public browse cards and sanctuary detail pages require instant access to `average_rating` and `total_reviews`. In high-traffic systems, calculating `AVG(rating)` on every page load causes disk I/O bottlenecks. Storing pre-aggregated rating metrics directly on `sanctuaries` provides O(1) read performance.
   Normalization: **Strictly 3NF compliant** with transactional updates handled by stored procedures and business logic.
5. **Consolidation of `booking_attendees` into `bookings` (Saved 1 Table)**
   Why: In airline, railway, and safari reservation engines, tickets track passenger volume via `number_of_people`, with the primary traveller identity verified via `users.user_id`. The passenger manifest is dynamically synthesized on the PDF boarding pass from the verified primary booker and party count.
   Normalization: **Strictly 1NF, 2NF, 3NF compliant** because seat capacity calculations and row-level locks operate on the integer seat count.

2. Core Business Logic Implementation & Code Defense

Logic 1: Preventing Double-Booking of Guides and Vehicles (Concurrency Defense)

Examiner Question: *"How do you guarantee that a guide or vehicle is not assigned to two different safari tours at the exact same time?"*

Implementation: WildBook enforces this at the **storage engine level** using **Compound UNIQUE Constraints** in `slots` table (`database/01_schema.sql`):

```
CONSTRAINT uq_slot_guide UNIQUE (guide_id, slot_date, slot_time),
CONSTRAINT uq_slot_vehicle UNIQUE (vehicle_id, slot_date, slot_time)
```

Defense: Application-level `if (!exists)` checks are vulnerable to concurrency race conditions when two operators schedule slots at the exact same millisecond. By delegating the guarantee to InnoDB compound unique indexes, the database rejects the second conflicting transaction with `ER_DUP_ENTRY`.

Logic 2: Overbooking Prevention & 10-Minute Cart Hold (Concurrency Locking)

Examiner Question: *"What happens if two visitors try to book the last 2 seats on the same safari slot simultaneously? How do you prevent overbooking?"*

Implementation: WildBook implements atomic capacity checks and seat reservation inside the MySQL stored procedure `sp_create_booking` (`database/03_procedures.sql`):

```
-- 1. Acquire exclusive Row-Level Lock on the slot to block concurrent reads
SELECT s.capacity INTO v_capacity
FROM slots s
WHERE s.slot_id = p_slot_id AND s.is_active = TRUE
FOR UPDATE;

-- 2. Calculate currently committed seats (CONFIRMED + unexpired PENDING holds)
SELECT COALESCE(SUM(b.number_of_people), 0) INTO v_booked
FROM bookings b
WHERE b.slot_id = p_slot_id
  AND (
    b.status = 'CONFIRMED'
    OR (b.status = 'PENDING' AND b.hold_expires_at > NOW())
  );

-- 3. Reject atomically if remaining seats are insufficient
IF (v_capacity - v_booked) < p_num_people THEN
  SET p_status_code = 'NOT_ENOUGH_SEATS';
  ROLLBACK;
ELSE
  -- Reserve seats with 10-minute hold
  INSERT INTO bookings (..., status, hold_expires_at)
  VALUES (..., 'PENDING', DATE_ADD(NOW(), INTERVAL 10 MINUTE));
  SET p_status_code = 'SUCCESS';
  COMMIT;
END IF;
```

Defense: The `FOR UPDATE` clause tells MySQL to place an exclusive write lock on that specific slot row. The second visitor's transaction is paused until the first finishes. If the first takes the last seats, the second transaction evaluates `(capacity - booked) < num_people`, safely receives `NOT_ENOUGH_SEATS`, and rolls back cleanly without dirty reads.

Logic 3: Automatic Hold Expiration (Ghost Booking Prevention)

Examiner Question: *"What if a visitor reserves seats but closes their browser without paying? Are those seats locked forever?"*

Implementation: Two-tier expiration strategy:

1. **Query-Time Filter**: All availability views (`v_slot_availability`) and booking creation procedures only treat a `PENDING` booking as active if `hold_expires_at > NOW()` . The instant 10 minutes pass, the seats automatically become available to other guests.
2. **Background Cleanup Event**: MySQL Event Scheduler runs `e_expire_pending_holds` every 1 minute executing stored procedure `sp_expire_pending_bookings` , transitioning stale holds to status `EXPIRED` .

Logic 4: Safari Cancellation Split (40% Platform Fee / 60% Refund)

Examiner Question: *"How does cancellation work, and where is the refund logic implemented?"*

Implementation: Managed by stored procedure `sp_cancel_booking` (`database/03_procedures.sql`):

```
-- Calculates exact 40% retention and 60% refund
SET v_fee_amount = ROUND(v_total_amount * 0.40, 2);
SET v_refund_amount = ROUND(v_total_amount * 0.60, 2);

-- Atomically updates booking record
UPDATE bookings
SET status = 'CANCELLED',
    cancelled_at = NOW(),
    cancellation_reason = 'Cancelled by guest',
    cancellation_fee = v_fee_amount,
    refund_amount = v_refund_amount
WHERE booking_id = p_booking_id;

-- Updates payment transaction ledger
UPDATE payments
SET payment_status = 'PARTIALLY_REFUNDED'
WHERE booking_id = p_booking_id;
```

3. Complete File-by-File Guide (Database, Backend & Frontend)

Database Tier (`/database`)

FILE	RESPONSIBILITY & CORE ARCHITECTURAL FUNCTION
`01_schema.sql`	Defines all 9 relational tables, column datatypes, primary keys, foreign key constraints (`ON DELETE RESTRICT`), and compound unique indexes (`uq_slot_guide` , `uq_slot_vehicle`).
`02_triggers.sql`	Implements relational integrity triggers: `trg_slot_capacity_check` (prevents slot capacity > vehicle capacity), `trg_slot_sanctuary_mismatch` (ensures activity, guide, and vehicle belong to the same reserve), and `trg_slot_admin_approved` (restricts slot creation to approved admins).
`03_procedures.sql`	Contains ACID stored procedures: `sp_create_booking` (pessimistic seat lock & 10-min hold), `sp_confirm_payment` (transitions PENDING → CONFIRMED), `sp_cancel_booking` (40%/60% cancellation split), and recurring cleanup event `e_expire_pending_holds` .
`04_views.sql`	Encapsulates analytical queries: `v_public_sanctuaries` (sanctuaries with cover image & price range), `v_slot_availability` (real-time remaining seats computation), and `v_confirmed_bookings` (joined visitor manifest).
`05_seed.sql`	Prepopulates realistic production data across all 9 tables: 8 wildlife reserves, verified activities, guides, vehicles, and booking histories.
`run_all.js`	Automated migration runner executing schema, triggers, procedures, views, seed data, and a 12-test automated assertion suite.

Backend Tier (`/backend`)

FILE	RESPONSIBILITY & CORE ARCHITECTURAL FUNCTION
`server.js`	Express application entry point, mounts CORS, JSON body parser, static file streaming (`/uploads`), and API route clusters.
`config/db.js`	MySQL2 connection pool manager with auto-reconnection and parameterized query execution.
`middleware/auth.js`	JWT authentication verification extracting user identity and role from `Bearer` token header.
`middleware/roleCheck.js`	RBAC role barrier middleware enforcing role authorization (`VISITOR` , `ADMIN` , `SITE_OWNER`) with HTTP 403 Forbidden on violations.
`controllers/authController.js`	Handles registration and authentication for all roles: visitor signup, admin operator registration with multipart PDF uploads, and owner login.
`controllers/bookingController.js`	Visitor booking lifecycle: calls `sp_create_booking` , fetches real-time hold seconds, streams in-memory PDF boarding tickets with QR codes, and executes cancellations.
`controllers/sanctuaryController.js`	Public sanctuary catalogue: fetches approved reserves from `v_public_sanctuaries` , search filtering, and detail page queries.
`controllers/adminSanctuaryController.js`	Admin sanctuary management: creates draft sanctuaries, validates submission prerequisites, uploads cover photos, and manages safari activities.
`controllers/adminBookingController.js`	Admin booking manifest: displays passenger lists with masked government ID numbers (e.g. `****1234`) and operational status filters.
`controllers/ownerController.js`	Site Owner administrative control: reviews pending operator applications, inspects uploaded PDF license proofs, and approves or rejects admins.
`controllers/ownerStatsController.js`	Platform analytical metrics: correlated subqueries computing slot occupancy percentages, monthly net revenues, and frequent visitor rankings.
`verify_workflow.js`	Comprehensive 25-step end-to-end multi-role test suite testing signup, login, draft sanctuary submission, approval, visitor booking, and tickets.

Frontend Tier (`/frontend/src`)

FILE	RESPONSIBILITY & CORE ARCHITECTURAL FUNCTION
`pages/user/Home.jsx`	Visitor landing page with hero banner, rotating featured reserves, eco-tourism trust badges, and quick search bar.
`pages/user/Sanctuaries.jsx`	Public reserve exploration: grid view of approved sanctuaries with starting prices, cover photos, and dynamic ratings.
`pages/user/SanctuaryDetail.jsx`	Individual sanctuary overview: exhibits reserve description, operating hours, guidelines, and available safari experiences.
`pages/user/Payment.jsx`	Checkout gateway: displays live 10-minute countdown timer, passenger manifest summary, and mock card payment submission.
`pages/user/Bookings.jsx`	Visitor dashboard: tabbed view of Upcoming, Past, and Cancelled bookings with one-click PDF boarding pass download and cancellation modal.
`pages/admin/Dashboard.jsx`	Sanctuary Admin operations overview: occupancy rates, upcoming slots, revenue charts, and quick-action navigation.
`pages/admin/SanctuaryForm.jsx`	Sanctuary registration and editing form with 1-click “Demo Prerequisites” autofill attaching sample cover photos, activities, guides, and vehicles.
`pages/admin/Signup.jsx`	Operator onboarding page featuring **Rotating 8-Operator Auto-fill** cycling through real government wildlife directors and attaching PDF proofs.
`pages/owner/AdminRequests.jsx`	Site Owner verification dashboard: inspects operator credentials, downloads submitted PDF license proofs, and approves or rejects applications.
`pages/owner/Stats.jsx`	Site Owner executive analytics: interactive occupancy heatmaps, frequent visitor rankings, and monthly platform revenue breakdowns.

4. Rotating Auto-Fill Demonstration Data Guide

To ensure smooth demonstrations without manual typing or colliding with existing database records, WildBook includes built-in rotation queues:

Admin Operator Registration Queue (8 Candidates)

1. **Dr. Ananya Roy** — Sundarbans Biosphere Reserve (Field Director)
2. **Vikramaditya Chauhan** — Jim Corbett National Park (Conservator of Forests)
3. **Kavitha Manjunath** — Bhadra Bio-Resilience Society (Deputy Conservator of Forests)
4. **Dr. Rajesh Kumar** — Kabini Wildlife Reserve (Field Director)
5. **Bikash Chandra Mohanty** — Similipal Biosphere Reserve (Deputy Conservator of Forests)
6. **Fateh Singh Bhati** — Thar Desert Wildlife Protection Trust (Divisional Forest Officer)
7. **Pemba Lepcha** — Khangchendzonga High-Altitude Sanctuary (Senior Range Officer)
8. **Dr. S. Karthikeyan** — Anamalai Rainforest Conservation Trust (Chief Naturalist & Trustee)

Each click of the auto-fill button rotates to the next candidate, automatically generates unique registration numbers and official emails, and attaches sample PDF credentials.

Visitor Registration Queue (6 Travellers)

1. **Aditi Rao** — `aditi.rao@wildbook.com`
2. **Rohan Verma** — `rohan.verma@wildbook.com`
3. **Sneha Kulkarni** — `sneha.kulkarni@wildbook.com`
4. **Vikramaditya Sen** — `vikramaditya.sen@wildbook.com`
5. **Maya Patel** — `maya.patel@wildbook.com`
6. **Arjun Nair** — `arjun.nair@wildbook.com`